TASK 1: Log File Analyzer

Domain: DevOps / Backend Monitoring | Level: Basic

What is this project?

A Log File Analyzer reads web server log files (Apache/Nginx), counts HTTP errors like 404 and 500, identifies the top IP addresses making requests, finds the most accessed pages, and saves a report in CSV or JSON format.

Key Concepts to Learn First

• File I/O in Java (BufferedReader, FileWriter)

• Regular Expressions (java.util.regex) – to extract patterns from text

• HashMap – to count occurrences (errors, IPs, endpoints)

• Collections.sort() – to rank top results

• Writing CSV/JSON output files

Step-by-Step Explanation

Step 1 – Read the log file line by line

Use BufferedReader to read large files efficiently without loading everything into memory.

Step 2 – Use Regex to parse each line

Apache Common Log Format looks like: 127.0.0.1 - - [10/Jun/2025] "GET /index.html HTTP/1.1" 200 1024
Use regex groups to capture: IP, method, URL, status code.

Step 3 – Count with HashMap

For each line, increment counters in HashMaps: errorCount.put(status, errorCount.getOrDefault(status,0)+1)

Step 4 – Sort and find Top-N

Convert HashMap to a List of entries, then sort by value descending.

Step 5 – Export report

Write results to a .csv file using FileWriter or to .json using manual string building or Gson library.

Complete Java Code

```
import java.io.*;
import java.util.*;
import java.util.regex.*;

/**
 * Log File Analyzer
 * Parses Apache/Nginx log files and generates a report.
 */
public class LogFileAnalyzer {

    // Apache Common Log Format pattern
```

```
// Example line: 127.0.0.1 - - [10/Jun/2025] "GET /page HTTP/1.1" 404 512
static final String LOG_PATTERN =
    "^(\S+) \S+ \S+ \[.*?\] \"(\S+) (\S+) \S+\" (\d{3}) (\d+|-)";

public static void main(String[] args) throws IOException {

    // ■■ Change this to your actual log file path ■■
    String logFile = "access.log";

    Map<String, Integer> errorCount   = new HashMap<>();
    Map<String, Integer> ipCount      = new HashMap<>();
    Map<String, Integer> endpointCount= new HashMap<>();
    int totalLines = 0;

    Pattern pattern = Pattern.compile(LOG_PATTERN);

    // ■■ Step 1: Read file line by line (memory-efficient) ■■
    try (BufferedReader br = new BufferedReader(new FileReader(logFile))) {
        String line;
        while ((line = br.readLine()) != null) {
            totalLines++;
            Matcher m = pattern.matcher(line);
            if (m.find()) {
                String ip       = m.group(1);  // e.g. 192.168.1.1
                String method   = m.group(2);  // e.g. GET
                String endpoint = m.group(3);  // e.g. /index.html
                String status   = m.group(4);  // e.g. 404

                // ■■ Step 2: Count HTTP errors (4xx, 5xx) ■■
                if (status.startsWith("4") || status.startsWith("5")) {
                    errorCount.merge(status, 1, Integer::sum);
                }

                // ■■ Step 3: Count IPs and endpoints ■■
                ipCount.merge(ip, 1, Integer::sum);
                endpointCount.merge(endpoint, 1, Integer::sum);
            }
        }
    }

    // ■■ Step 4: Print Summary ■■
    System.out.println("=== LOG ANALYSIS REPORT ===");
    System.out.println("Total Lines Processed: " + totalLines);

    System.out.println("\n--- HTTP Errors ---");
    errorCount.entrySet().stream()
        .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
        .forEach(e -> System.out.println("  " + e.getKey() + " : " + e.getValue() + " times"));

    System.out.println("\n--- Top 5 IP Addresses ---");
    ipCount.entrySet().stream()
        .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
        .limit(5)
        .forEach(e -> System.out.println("  " + e.getKey() + " : " + e.getValue() + " requests"));

    System.out.println("\n--- Top 5 Endpoints ---");
    endpointCount.entrySet().stream()
        .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())
        .limit(5)
```

```
                .forEach(e -> System.out.println("  " + e.getKey() + " : " + e.getValue() + " hits"));

            // ■■ Step 5: Export to CSV ■■
            exportCSV(errorCount, ipCount, endpointCount);
            System.out.println("\nReport saved to: report.csv");
        }

        // Writes results to a CSV file
        static void exportCSV(Map<String,Integer> errors,
                              Map<String,Integer> ips,
                              Map<String,Integer> endpoints) throws IOException {
            try (FileWriter fw = new FileWriter("report.csv")) {
                fw.write("Type,Key,Count\n");
                for (var e : errors.entrySet())
                    fw.write("Error," + e.getKey() + "," + e.getValue() + "\n");
                for (var e : ips.entrySet())
                    fw.write("IP," + e.getKey() + "," + e.getValue() + "\n");
                for (var e : endpoints.entrySet())
                    fw.write("Endpoint," + e.getKey() + "," + e.getValue() + "\n");
            }
        }
    }
```

Sample Log File (access.log) to Test With

```
192.168.1.1 - - [10/Jun/2025] "GET /index.html HTTP/1.1" 200 1024
192.168.1.2 - - [10/Jun/2025] "GET /about HTTP/1.1" 404 512
192.168.1.1 - - [10/Jun/2025] "POST /login HTTP/1.1" 500 256
10.0.0.1   - - [10/Jun/2025] "GET /index.html HTTP/1.1" 200 1024
192.168.1.2 - - [10/Jun/2025] "GET /contact HTTP/1.1" 404 300
```

Expected Output

```
=== LOG ANALYSIS REPORT ===
Total Lines Processed: 5

--- HTTP Errors ---
  404 : 2 times
  500 : 1 times

--- Top 5 IP Addresses ---
  192.168.1.1 : 2 requests
  192.168.1.2 : 2 requests
  10.0.0.1    : 1 requests

--- Top 5 Endpoints ---
  /index.html : 2 hits
  /about      : 1 hits

Report saved to: report.csv
```

How to Run

```
# Step 1: Save code as LogFileAnalyzer.java
# Step 2: Create access.log with sample data above
# Step 3: Compile
  javac LogFileAnalyzer.java

# Step 4: Run
  java LogFileAnalyzer
```

TIP: What you'll learn from this task

Regex parsing • HashMap counting • Stream sorting • File I/O • CSV export